# Dynamical properties of the Hopfield model through bitwise arithmetic

Bastien Dumont,[1,2] David Lacoste,[3] and Michele Castellana[4]

[1] *Aix-Marseille Université, Marseille, France*
[2] *Université Paris-Saclay, Orsay, France*
[3] *Gulliver Laboratory, CNRS UMR 7083, ESPCI Paris, PSL Research University, Paris, France*
[4] *Institut Curie, PSL Research University, CNRS UMR 168, Paris, France*

(Dated: November 3, 2026)

The Hopfield model, introduced in its now-canonical form by J.J. Hopfield in 1982, laid the foundation for the understanding of neural networks, whether they are biological or artificial. This model has become a reference in the statistical mechanics of networks and its equilibrium properties have been extensively studied and are now well understood. Nevertheless, much remains to be discovered about the dynamics leading to those equilibrium properties. Studying such dynamical properties requires the use of optimized algorithms capable of probing large samples of trajectories or initial conditions, as well as potentially rare events. In this internship project, we use a bitwise implementation of the Metropolis algorithm to study the properties of several discrete-value network models such as the Ising model, the Edwards-Anderson model and the Hopfield model. This implementation takes advantage of the native machine-specific binary arithmetic to efficiently simulate the evolution of $n_{\text{bits}}$ systems in parallel. We show that this approach is well-adapted to simulate discrete value models, and in particular the Hopfield network, and can also yield significant computational gains when properly implemented at the low level.

## I. INTRODUCTION

### A. Subject

In October 2024, John J. Hopfield was awarded the Nobel Prize in Physics alongside Geoffrey Hinton for their pioneering work on artificial neural networks [1]. Their work laid the foundations for further development and understanding of neural networks, whether they are organic or artificial, and eventually gave rise to Artificial Intelligence (AI).

Hopfield worked on an artificial neural network originally invented by Shun'ichi Amari in 1972 [2]. He greatly contributed to the analysis, understanding and popularization of the network in his landmark paper *Neural networks and physical systems with emergent collective computational abilities* published in 1982 [3], and the network was eventually named after him. The model, described in more details in subsection II A, is similar to the Ising and the spin glass networks: it is composed of a set of spins – called neurons in the case of the Hopfield network – $S_i \in \{-1, 1\}$ interacting with one another. In the same way as the two preceding models, the Hopfield model is energy-based, that is it will tend to minimize its energy over time. What makes this model different from the other two is the way the interactions between neurons are defined: as for the Ising and spin glass models they remain static over time but depend on the set of patterns to be stored in the network, making the latter attractors, that is to say minima of energy, of the dynamics.

The Hopfield model provided the first simple yet powerful framework for understanding networks of interconnected neurons. It has become a reference in statistical mechanics on networks and its properties at equilibrium have been extensively studied and are now well known, whether it is its statistical mechanics [4–7] or its phase diagram and transitions [8]. However, much less is known about the dynamics that lead to those well-known equilibrium properties.

### B. Objectives

The main objective of this internship is to efficiently simulate the Metropolis dynamics on the Hopfield network. To this end, we use a bitwise formalism that allows us to simulate several replicas of the system in parallel, increasing the efficiency of the exploration of the model's energy landscape. Our objectives in this internship are thus twofold: first, to validate the bitwise implementation and quantify its computational gain in yield over the standard Metropolis algorithm; second, to leverage this implementation to study the dynamics of the Hopfield network. To validate the implementation, we begin with the Ising model, which serves as a natural benchmark due to its simplicity and well-characterized phase transition. Once this implementation has been proven sane and efficient on the Ising network, we increase the complexity by implementing it on the spin glass network, testing the gain in yield again.

Once the bitwise arithmetic has been validated on both models, we implement it on the Hopfield network in order to analyze its dynamics.

## II. THE HOPFIELD MODEL

We here present the Hopfield model, which is of interest in this internship.

### A. The model

The Hopfield Hamiltonian resemble this of the spin glass model and is as follows:

$$H_{\text{Hopfield}} = -\frac{1}{2}\sum_{i\neq j} w_{ij} S_i S_j, \tag{1}$$

where in this case the weights $w_{ij}$ are not initialized randomly but rather determined by a set of states that one wants to "store" in the network (this term will be explained further on):

$$w_{ij} = \frac{1}{N}\sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu. \tag{2}$$

With $\{\xi^1, \xi^2 \ldots, \xi^p\}$ the set of patterns to be memorized by the network, each pattern being a state of the system with components $\xi_i^\mu = \pm 1$. This initialization follows the Hebb rule "neurons that fire together wire together" [9]. By setting the weights this way, the energy landscape is shaped with local minima at each pattern coordinates.

By initializing the network not far from one of the stored patters, the dynamics will make it tend to the precise pattern's configuration, given that we are located in the memory phase of the model. In this way, the network is said to have associative memory: it can recover a stored pattern from a partial or noisy version of that pattern. A visual representation of the model is shown in **??**.

A parameter of particular interest in this model is its storage load, defined as the number of patterns to be stored over the size of the network: $\alpha = p/N$ with p the number of patterns to be stored. The equilibrium properties of the model, depending both on the temperature and on this storage load have been extensively studies in [4–7]. We show in **??** the phase diagram of the Hopfield model on a fully-connected network, which is the reference topology of the model. In the limit $T \to 0$, the model is known to exhibit a phase transition at $\alpha_c = 0.138$. This critical storage load value can be regarded as the network's storage capacity, as beyond this threshold the model is no longer able to reconstruct the stored data from partial or corrupted data. Above the network's capacity, the energy minima overlap, leading to mixed or spurious (parasite) states that prevent accurate retrieval of stored information.

In order to study the capacity of the network to recover stored data, we introduce the overlap order parameter:

$$m^\mu = \frac{1}{N}\sum_{i=1}^{N} \xi_i^\mu S_i. \tag{3}$$

This quantity measures how much the system's configuration is aligned with the pattern $\mu$. Using the definition of $m^\mu$, the energy of the system can be re-written:

$$E = -\frac{N}{2}\sum_{\mu=1}^{p} (m^\mu)^2. \tag{4}$$

All the $m^\mu$ are of order $1/\sqrt{N}$ for a state that is uncorrelated with the memories, while, in the memory phase, $m^*$ should be of the order of the unit [7]. Thus, we often consider the pattern for which the overlap is maximum: $m^* = \max_\mu |m^\mu|$. We take the absolute value of the overlap as the system can also end up in the configuration opposite of the stored pattern due to the $\mathbb{Z}_2$ global spin flip symmetry. From $m^*$ we can define the reconstruction error of the network:

$$\text{error} = \frac{1 - m^*}{2} \tag{5}$$

We show in Figure 1 the evolution of the error with the storage load:

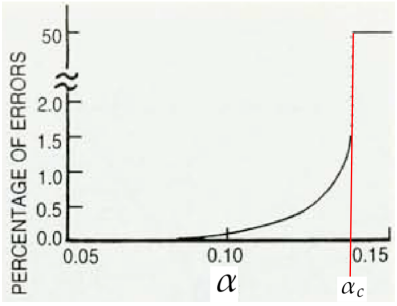



Figure 1: Evolution of the error with the storage load, extracted from [7]. For this diagram, the configuration is initialized near a pattern. When the storage load exceeds $\alpha_c$, the configuration becomes totally uncorrelated with the stored patterns and the error suddenly jumps to 0.5.

## III. METHOD

### A. The Metropolis algorithm

In order to make the models evolve in time, which is necessary to observe their dynamics, we use the well-known metropolis algorithm [10]. For models with a defined Hamiltonian, the Metropolis algorithm proceeds as follows:

1. Start from an initial configuration $\{S_i\}$.
2. Repeat $N$ times:
   a. Pick a random spin $S_k$ and compute the energy change $\Delta E_k$ upon flipping it.
   b. If $\Delta E_k \leq 0$, accept the flip.
   c. Else, draw a random number $r \sim \mathcal{U}([0,1])$ and accept the flip if $r \leq e^{-\beta \Delta E_k}$.
3. Repeat Step 2 for $N_{\text{sweeps}}$ sweeps.

Where $N$ is the number of spins, $\beta$ is the inverse temperature, and $N_{\text{sweeps}}$ is the number of sweeps, where one

sweep consists of $N$ successive single-spin flip attempts. This algorithm however becomes computationally demanding when one attempts to sample a large number of configurations. This situation occurs, for example, when looking for rare samples in the dynamics, or sampling many different initial conditions – both of which we will try to explore during this internship. We thus need to find way of optimizing the implementation of the algorithm.

### B. First optimization: the bitwise arithmetic

The first optimization in the implementation of the Metropolis algorithm is the use of bitwise arithmetic. The main idea is to leverage the boolean representation as stored in our computers. The typical object that we manipulate is called `Bits`: it is a collection of $n_{\text{bits}} = 64$ boolean variables. In that way, we have the possibility to simulate 64 evolutions of the same variable in parallel. This structure is very convenient to represent binary

$$W = \underbrace{\boxed{b^0}\boxed{b^1}\boxed{b^2}\cdots\boxed{b^{62}}\boxed{b^{63}}}_{64 \text{ bits} \leftrightarrow 64 \text{ replicas}}$$

Figure 2: A `Bits` $W$ encoding 64 parallel replicas. Each bit $b^k$ carries the state of replica $k$.

variable such as the spins of the system. The boolean representation $b^k \in \{0, 1\}$ of a spin $s^k \in \{-1, +1\}$ is given by $b^k = (s^k + 1)/2$. In that way, we are able to implement several replicas of the full spin system in a set of `Bits` $\{S_i\}$, each encoding the replicas of the associated spin, as shown in Figure 3. Storing all replicas in a single `Bits` object allows us to manipulate and update them simultaneously using a single bitwise operator. This is an instance of SWAR (SIMD Within A Register) parallelism, where SIMD stands for Single Instruction, Multiple Data: all 64 replicas are updated by a single bitwise instruction. The restriction to $n_{\text{bits}} = 64$ reflects the native word size of modern 64-bit CPUs [11]. This could in principle be extended to 512 bits using AVX-512 SIMD instructions, at the cost of introducing a new register type in the implementation.
The convention used for what follows is presented in the Table I:

| Object | Notation |
|---|---|
| $n_{\text{bits}}$ replicas of spin i | $S_i \in \{-1, 1\}^{n_{\text{bits}}}$ |
| $n_{\text{bits}}$-bit word (`Bits`) | $B_i \in \{0, 1\}^{n_{\text{bits}}}$ |
| Replica $k$ of spin $i$ | $s_i^{(k)} \in \{-1, +1\}$ |
| Replica $k$ of bit $i$ | $b_i^{(k)} \in \{0, 1\}$ |

Table I: Notation for the bitwise representation of spins.

Now that we know how to conveniently handle binary variables such as spins, we need to employ the same formalism to represent integer, such as the sum $\Delta E_k$.

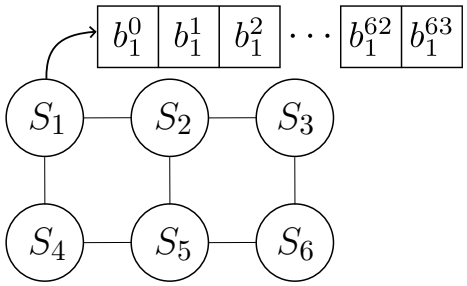



Figure 3: Bitwise implementation of a spin system. A given replica of the entire system is obtained by selecting the corresponding replica in each spin.

This is done using a `BitSet`, which is a collection of `Bits`. When each `Bits` is associated with a power of 2, the `BitSet` represent a collection of $n_{\text{bits}}$ unsigned integers and the `BitSet` is called `UnsignedInt`. We show an example of `UnsignedInt` in the Figure 4. In the same

$$Z = \underbrace{\begin{array}{l} \boxed{b_0^0}\boxed{b_0^1}\boxed{b_0^2}\cdots\boxed{b_0^{62}}\boxed{b_0^{63}}\ 2^0 \\ \boxed{b_1^0}\boxed{b_1^1}\boxed{b_1^2}\cdots\boxed{b_1^{62}}\boxed{b_1^{63}}\ 2^1 \\ \vdots \qquad\qquad\qquad\qquad\qquad \vdots \\ \boxed{b_l^0}\boxed{b_l^1}\boxed{b_l^2}\cdots\boxed{b_l^{62}}\boxed{b_l^{63}}\ 2^l \end{array}}_{64 \text{ replicas}}$$

Figure 4: An `UnsignedInt` encoding 64 parallel unsigned integers. Each row is a `Bits` associated with a power of 2. The value of each integer is obtained by reading the `UnsignedInt` along the corresponding column. For instance, the fifth integer stored in $Z$ is $Z_4 = \sum_{i=0}^{l} b_i^4\, 2^i$.

way as for `Bits`, the `UnsignedInt` data structure enables us to add, subtract or even multiply two sets of 64 integers with just the cost of one operation.
This implementation method has proven efficient when dealing with discrete-value models such as the Ising [12] or the spin glass [13] models, but it has not yet been employed to study the Hopfield network.

### C. Second Optimization: Shared random numbers

The bitwise implementation is a structural optimization of the data representation. On top of this structural optimization can be added a physical one which consists in having all replicas share the same sequence of random numbers, fully parallelizing their evolution. This turns

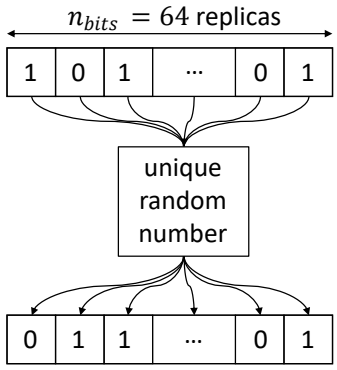



Figure 5: All 64 replicas of a given spin share the same random number at each step.

out to be central: random number generation is often the simulation bottleneck, as it involves costly operations such as multiplications and is called at every step of the hot loop. A schematic view of the shared random number is shown in Figure 5. Using the same random number among all evolutions raises the questions of the independence of the realizations, this will be the subject of the next subsection.

### D. Note on the independence of the realizations

Because all replicas share the same sequence of random number, it is not obvious that their evolutions are independent. This is in fact guaranteed by two facts. The first one is that each replica is initialized independently of the others. The second one is that the shape of the energy landscape in each realization is also shaped independently of the others. Thus, the shared random number yields different outcome in each replica. A schematic view of this mechanism is shown in Figure 6. In a further analysis, one would be tempted to share the energy landscape across all replicas. This happens for instance if all replicas share the same patterns to be stored in the case of the Hopfield model, see subsection II A. In this case, the realizations would be correlated through this shared energy landscape.

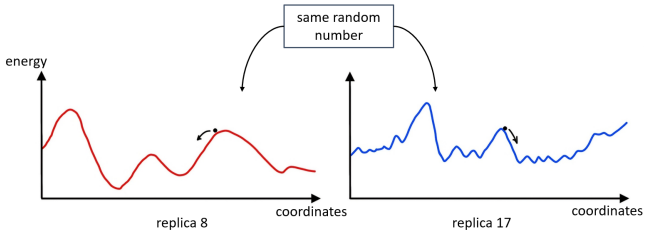



Figure 6: Schematic view of the independence of the realizations evolving from the same sequence of random numbers. Because of the unique energy landscape in every replica and the independent initializations, the same random number yields independent evolutions.

### E. Standard implementation of the Metropolis algorithm

We show here the standard implementation of the Metropolis algorithm, which will be broadly shared across all models.
If $\Delta E_k \leq 0$ the flip is accepted. Else, the flip condition $r \leq e^{-\beta \Delta E_k}$, yields:

$$\begin{aligned} -\frac{1}{\beta}\log r &\geq \Delta E_k \\ -\frac{1}{\beta\kappa}\log r &\geq \Delta_k, \end{aligned} \tag{6}$$

with

$$\Delta_k := \frac{\Delta E_k}{\kappa}. \tag{7}$$

where $\kappa$ corresponds to the constants of the model (*i.e.* $2J$ in the case of the Ising model or $2/N$ for the Hopfield model). With this normalization, $\Delta_k$ is an integer and can be expressed using `UnsignedInts`. With $r \sim \mathcal{U}([0,1])$, we know that $-\log r \sim \mathcal{E}(1)$ , thus:

$$\theta := -\frac{1}{\beta\kappa}\log r \sim \mathcal{E}(\beta\kappa). \tag{8}$$

The left-hand side of Equation (6) is a random variable of probability density $p(\theta) = \beta\kappa e^{-\beta\kappa\theta}$ and mean $(\beta\kappa)^{-1}$. $\Delta_k$ is an integer and is guaranteed to be positive by the first escape condition of the algorithm. We can thus consider only its integer part without changing the inequality, which yields an integer-only flip condition, defining $q := \lfloor\theta\rfloor$:

$$q \geq \Delta_k. \tag{9}$$

A first naive implementation of the classical algorithm is given in the **??** in the appendix, where each of the $n_{\text{bits}}$ branch possesses its own random number. A more efficient implementation would be to share the drawn random threshold across the replicas. This shared random number version yields a fast escape condition, taking advantage of the inequality

$$\Delta_k \leq \Delta_k^{\max},$$

where $\Delta_k^{\max}$ only depend on the topology of the network. Plugging this into Eq (9) yields the flip escape condition:

$$q \geq \Delta_k^{\max} \tag{10}$$

When satisfied, this guarantees that Eq. (9) holds for every replica regardless of its local configuration, so that all replicas are flipped simultaneously without computing the per-replica test. This condition is typically met in the high-temperature regime, where $\theta$ is more likely to be large.
We show in the Algorithm 1 the shared random number

**Algorithm 1:** Standard Metropolis implementation

**Input:** Lattice spins $\{S_i\}$, coupling $J$, inverse temperature $\beta$
**Output:** Updated lattice spins $\{S_i\}$

**for** sweep $= 1$ **to** $N_{\text{sweeps}}$ **do**
  **for** $k = 1$ **to** $N$ **do**
    Draw $q$ with $\theta \sim \mathcal{E}(\beta\kappa)$
    **if** $q \geq \Delta_k^{max}$ **then**
      **for** $r = 1$ **to** $n_{bits}$ **do**
        $s_k^{(r)} \leftarrow -s_k^{(r)}$
    **else**
      **for** $r = 1$ **to** $n_{bits}$ **do**
        Compute $\Delta_k^{(r)}$
        **if** $\Delta_k^{(r)} \leq 0$ ***or*** $q \geq \Delta_k^{(r)}$ **then**
          `// the test` $\Delta_k^{(r)} \leq 0$ `is not necessary as` $q$ `has already been drawn, but is shown for pedagogy`
          $s_k^{(r)} \leftarrow -s_k^{(r)}$

implementation of the Metropolis algorithm. This implementation is valid for all models, provided that the definition of $\kappa$ is adapted to each model.

This reduces the number of random draws per $n_{\text{bits}}$ cycle update to just one, shared among all replicas. This optimization is possible even though the structure is not fully parallelized across replicas. The only case for which this parallelized random number optimization might not be as efficient as expected is when $\Delta_k^{(r)} \leq 0$ for all replicas simultaneously, in which case a random threshold is drawn but never used. The complexity of this algorithm on a given network is $\mathcal{O}\left(N_{\text{sweeps}} \cdot N \cdot \langle n_{\text{neighbor}} \rangle \cdot n_{\text{bits}}\right)$.

### F. Bitwise Implementation of the Metropolis algorithm on the Hopfield network

Given the Hamiltonian (1), the energy shift when flipping the neuron $S_k$ is:

$$\Delta E_k = 2 \sum_{i \partial k} w_{ki} S_k S_i \tag{11}$$

We write $w_{ki} = \frac{1}{N} G_{ki}$ in order to deal with integer-only variables. Thus:

$$\frac{\Delta E_k N}{2} = \Delta_k = \sum_{i \in \partial k} G_{ki} S_i S_k \qquad \text{with} \quad G_{ki} \in [-p, p].$$

And the flip condition becomes:

$$q \geq S_k h_k, \tag{12}$$

with $h_k = \sum_{i \in \partial k} G_{ki} S_i$ the local field and $q = \lfloor \theta \rfloor = -\frac{N}{2\beta} \log r$.

The integer factor $G_{ki}$, is suitably stored into an `UnsignedInt` in the bitwise implementation. We operate the shift $\tilde{G}_{ki} = p + G_{ki}$, $\tilde{G}_{ki} \in [0, 2p]$ in order to deal with positive only integers, as required by the `UnsignedInt` format. Then:

$$\begin{aligned} \sum_{i \in \partial k} G_{ki} S_i S_k &= \sum_{i \in \partial k} (\tilde{G}_{ki} - p)(2C_{ik} - 1) \\ &= -\sum_{i \in \partial k} \tilde{G}_{ki} + 2 \sum_{i \in \partial k} C_{ik} (\tilde{G}_{ki} - p) + n_k p, \end{aligned}$$

with $C_{ki} = B_k \odot B_i$. The bitwise flip test is then:

$$q + \sum_{i \in \partial k} \tilde{G}_{ki} + 2p \sum_{i \in \partial k} C_{ki} \geq p n_k + 2 \sum_{i \in \partial k} C_{ki} \oplus \tilde{G}_{ki} \tag{13}$$

Where $C_{ki} \oplus \tilde{G}_{ik}$ consists in the XOR operator between $C_{ki}$ and every `Bits` composing the `UnsignedInt` $\tilde{G}_{ki}$:

$$\left(C_{ki} \oplus \tilde{G}_{ki}\right)_l = C_{ki} \oplus \left(\tilde{G}_{ki}\right)_l,$$

where $l$ denotes the $l$-th `Bits` of $\tilde{G}_{ki}$ (associated with the power $2^l$). This is strictly equivalent to setting the columns of $\tilde{G}_{ik}$ to 0 where $C_{ki}$ is zero and leave the other ones untouched. This method thus requires the creation of several `UnsignedInt` objects, whose sizes are fixed throughout the evolution of the system. Also, $\tilde{G}_{ki}$ is simply determined by the pattern initialization of the system and does not depend on the spin configuration. It can thus be computed once before letting the system evolve, saving significant computation time. Looking at Equation (12), the escape condition that enables us to flip all replicas of the spin $k$ without considering each configuration individually becomes:

$$q \geq p\, n_k. \tag{14}$$

This bound only depends on the architecture of the network, and is generally much larger than the maximum value of $\Lambda_k$ actually reached in any of the replicas. A tighter bound is obtained by using the actual patterns stored in each of the $n_{\text{bits}}$ replicas:

$$q \geq \max_r \Big( \sum_{i \in \partial k} \left| G_{ki}^{(r)} \right| \Big). \tag{15}$$

We call this new threshold $\Delta_k^{esc}$. It is typically smaller than the worst-case one (by a factor of order $\sqrt{p}$, à vérifier), since the couplings $G_{ki}$ are rarely of order $p$. Lowering the escape threshold makes the escape condition trigger more often, making the algorithm faster.

We show in Algorithm 2 the bitwise implementation of the Metropolis algorithm on the Hopfield network, with LHS and RHS referring to the left and right-hand sides of eq (13).

**Algorithm 2:** Bitwise Hopfield Metropolis

**Input:** Bit-replicated spins $\{B_i\}$, couplings `Couplings`, neighbor lists $\partial i$, number of patterns $P$

**Output:** Updated spin configurations $\{B_i\}$

**Preallocated variables:** `Bits` $c_{ij}$, `mask` `UnsignedInt` $\Sigma_c$, $\Sigma_{cg}$, $\Sigma_g$, LHS, RHS, $tmp$

**for** $i = 1$ **to** $N$ **do**
  $\Sigma_g(i) \leftarrow 0$
  **for** $j \in \partial i$ **do**
    $\Sigma_g(i)$ += $G_{ij}$

`// Precompute coupling sums`

**for** sweep $= 1$ **to** $N_{\text{sweeps}}$ **do**
  **for** step $= 1$ **to** $N$ **do**
    Draw a random neuron $i$
    Draw $q$ with $\theta \sim \mathcal{E}(2\beta/N)$
    **if** $q \geq \Delta_k^{esc}$ **then**
      $B_i \leftarrow \neg B_i$ `// Unconditional flip`
    **else**
      $\Sigma_c \leftarrow 0$
      $\Sigma_{cg} \leftarrow 0$
      **for** $j \in \partial i$ **do**
        $c_{ij} \leftarrow B_i \odot B_j$ `// Replica-wise agreement`
        $\Sigma_c$ += $c_{ij}$
        $tmp \leftarrow G_{ij}$
        $tmp \leftarrow tmp \wedge c_{ij}$
        $\Sigma_{cg}$ += $tmp$
      RHS $\leftarrow P\, n_i + 2\,\Sigma_{cg}$
      LHS $\leftarrow \Sigma_g(i) + r + 2P\,\Sigma_c$
      `mask` $\leftarrow$ (RHS $\leq$ LHS)
      `// Replica-wise acceptance test`
      $B_i \leftarrow B_i \oplus$ `mask`

*1. Reducing complexity in dense networks using overlaps*

In the case of a fully-connected network, summing over the neighbors of $k$ amounts to sum over the entirety of the network, yielding a complexity of $\mathcal{O}(N)$ in itself. In order to reduce this complexity, one can also make use of the overlap to compute the neuron flip condition. We define the extensive overlap:

$$M^\mu = N m^\mu = \sum_{i=1}^{N} \xi_i^\mu S_i. \tag{16}$$

Then,

$$\begin{aligned} S_k h_k &= \sum_{i \in \partial k} G_{ki} S_i S_k \\ &= \sum_{\mu=1}^{p} \xi_k^\mu S_k \left( M^\mu - \sum_{\substack{i \notin \partial k \\ i \neq k}} \xi_i^\mu S_i - \xi_k^\mu S_k \right) \\ &= \sum_{\mu=1}^{p} \xi_k^\mu S_k M^\mu - \sum_{\mu=1}^{p} \sum_{\substack{i \notin \partial k \\ i \neq k}} \xi_k^\mu \xi_i^\mu S_k S_i - \sum_{\mu=1}^{p} \left(\xi_k^\mu S_k\right)^2. \end{aligned}$$

The second sum is simply the sum $G_{ki} S_i S_k$ over the non-neighbors of $k$. Each term of the last sum is equal to 1. The flip condition is thus:

$$q + p \geq \sum_{\mu=1}^{p} \xi_k^\mu S_k M^\mu - \sum_{\substack{i \notin \partial k \\ i \neq k}} G_{ki} S_i S_k. \tag{17}$$

The bitwise implementation of this condition requires that all quantities are positive. We thus define the shifted extensive overlaps $\tilde{M}^\mu = M^\mu + N$, $\tilde{M}^\mu \in [0, 2N]$. Then, writing $S_k \xi_k^\mu = 2C_k^\mu - 1$ with $C_k^\mu = B_k \odot X_k^\mu$, $X_k^\mu$ being the binary counterpart of $\xi_k^\mu$,

$$\begin{aligned} \sum_{\mu=1}^{p} \xi_k^\mu S_k M^\mu &= \sum_{\mu=1}^{p} \xi_k^\mu S_k (\tilde{M}^\mu - N) \\ &= -N \sum_{\mu=1}^{p} (2C_k^\mu - 1) + \sum_{\mu=1}^{p} (2C_k^\mu - 1)\, \tilde{M}^\mu \\ &= Np - 2N \sum_{\mu=1}^{p} C_k^\mu + 2 \sum_{\mu=1}^{p} C_k^\mu \oplus \tilde{M}^\mu - \sum_{\mu=1}^{p} \tilde{M}^\mu. \end{aligned}$$

We also use the previous decomposition of $G_{ki} S_i S_k$:

$$\sum_{\substack{i \notin \partial k \\ i \neq k}} G_{ki} S_i S_k = -\sum_{\substack{i \notin \partial k \\ i \neq k}} \tilde{G}_{ki} + 2 \sum_{\substack{i \notin \partial k \\ i \neq k}} C_{ik} \oplus \tilde{G}_{ki} - 2p \sum_{\substack{i \notin \partial k \\ i \neq k}} C_{ik} + (N - n_k - 1)p.$$

The overall bitwise flip condition is thus:

$$q + 2N \sum_{\mu=1}^{p} C_k^\mu + \sum_{\mu=1}^{p} \tilde{M}^\mu + 2 \sum_{\substack{i \notin \partial k \\ i \neq k}} C_{ik} \oplus \tilde{G}_{ki} \geq p n_k + 2 \sum_{\mu=1}^{p} C_k^\mu \oplus \tilde{M}^\mu + \sum_{\substack{i \notin \partial k \\ i \neq k}} \tilde{G}_{ki} + 2p \sum_{\substack{i \notin \partial k \\ i \neq k}} C_{ik}. \tag{18}$$

This flip condition only requires summing over the patterns and over the non-neighbors of the considered neuron,

rather than over its full neighborhood. In a dense network, where $|\partial k| \gg p$, this is significantly faster than the direct computation, reducing the computational cost from $\mathcal{O}(N)$ to $\mathcal{O}(P + |\overline{\partial k}|) = \mathcal{O}(\alpha N + |\overline{\partial k}|)$. The downside of this method is that it requires another summation on the patterns after the flip condition computation to update the overlaps, if the neuron should be flipped: $\tilde{M}^\mu \leftarrow \tilde{M}^\mu - 2\xi_k^\mu S_k$, with the bitwise equivalent: $\tilde{M}^\mu \leftarrow \tilde{M}^\mu + \text{mask}_k(2 - 4C_k^\mu)$ with $\text{mask}_k$ the `Bits` object obtained from the computation of the flip condition across the $n_{\text{bits}}$ replicas. This implementation is thus faster than the original one when $\rho \geq \alpha + 1/2$ with $\rho$ the density of the network. In the case of a fully-connected network, the flip condition (18) becomes:

$$q + 2N \sum_{\mu=1}^{p} C_k^\mu + \sum_{\mu=1}^{p} \tilde{M}^\mu \geq p(N-1) + 2 \sum_{\mu=1}^{p} C_k^\mu \oplus \tilde{M}^\mu, \tag{19}$$

and no summation over the non-neighbors is required. We present this implementation in **??**.

## IV. RESULTS

### A. Independence of the realizations

### B. Capacity of the network

A first validation test of the implementation is to reproduce the evolution of the reconstruction error shown in Figure 1. To this end, we choose the simplest case that is to initialize the system directly on one of the stored patterns and checking whether the network is able to retain the memory of that pattern throughout its evolution. This avoids having to fix, as an additional parameter, the initial distance between the system and the stored pattern. The model is initialized on a $32 \times 32$ fully connected square lattice. Each replica possess its own independend stored patterns. We fix the temperature at $\beta = 5.0$ in order to approach the $T \to 0$ limit case and we let the system evolve for $2^{14}$ sweeps, averaging on the second half of the evolution. We show in Figure 7 the obtained reconstruction error depending on $\alpha$. We observe the expected evolution, with a null reconstruction error for the lowest $\alpha$ and a burst increase around the critical value before a slowly increasing plateau, where the system fails to retain the memory of the initially stored pattern. The increase is steeper for the largest systems, the finite size effects being less and less significant. In this regime, the plateau value tends to increase with the lattice size, approaching the expected reconstruction error of 0.5, obtained when the global orientations of the neurons are totally uncorrelated with that of the stored patterns. The first explanation for this observation is that initializing the network on the exact pattern favors better retrieval. This explanation is however not robust to the increase of the plateau with the storage load. The second is that finite size effects favor a better random alignment, as the reconstruction error is computed based on the best overlap, and that smaller systems favors extreme value, the noise on each overlap being of the order of $\sim 1/\sqrt{N}$. The increase of the plateau with the storage load seems logical: the more patterns are stored, the less efficient the retrieval of any single one becomes.

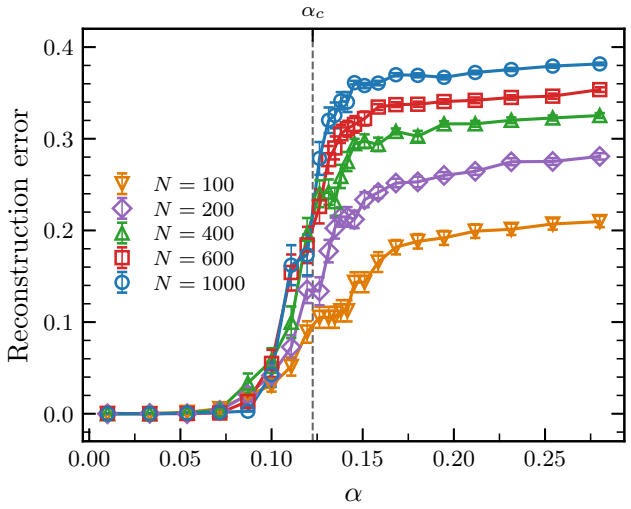



Figure 7: Evolution of the reconstruction error with the storage load for several lattice sizes. For this diagram, the configuration is initialized on one of the stored patterns.

### C. Dynamics visualization: initialization from a random state

We now aim at visualizing the evolution of the configurations when initializing the system from a random state. Again, the model is built on a $32 \times 32$ fully connected square lattice and each replica has its own saved patterns and initial state. The temperature is still set to $\beta = 5.0$. We give the evolution of $m^\mu$ for all patterns for a given replica in Figure 8. At the beginning, the state is roughly uncorrelated with any of the patterns. As the system evolves, it starts correlating with some patterns, before it eventually aligns perfectly ($m^\mu = -1$) with one of them. When it does so, the overlaps with the other patterns collapse to 0, with a noise of standard deviation 1/32. This configuration looks stable (as we can see small fluctuations around it), which was expected has it corresponds to a local minimum of energy.

We can also look at the evolution of the maximum overlap $m^*$ for all replicas at a given value of the storage load, as shown in Figure 9. While in some case the state quickly falls into a stable minimum of energy corresponding to

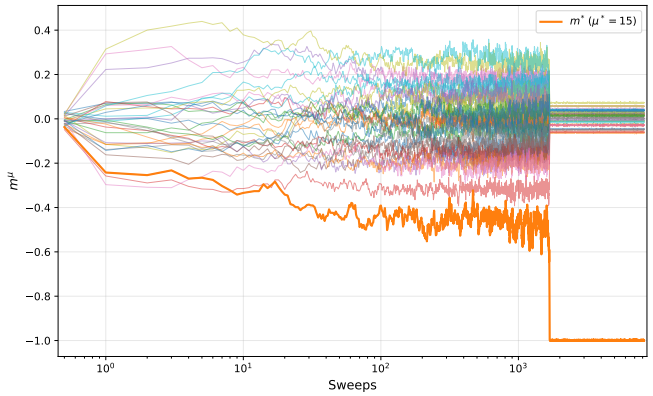



Figure 8: Evolution of $m^\mu$ for all patterns for the replica 16 of $\alpha = 0.03$.

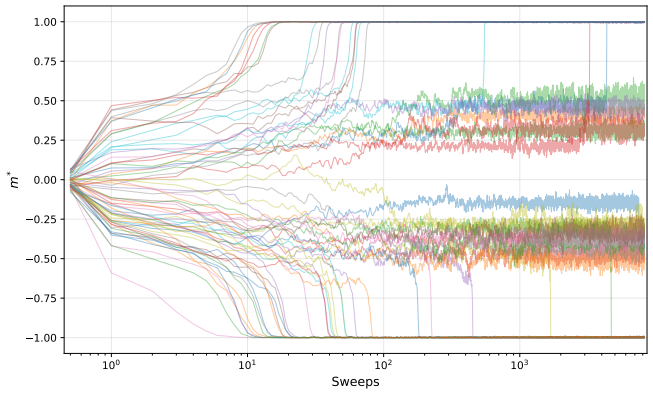



Figure 9: Evolution of the maximum overlap of each replica for $\alpha = 0.03$.

a specific pattern, some others seem to fluctuate around spurious states, for which the overlap is nor 1 or 0. While most configurations remain stuck in those spurious states (for the evolution time considered here), some others eventually escape those states to align with a specific pattern. The evolution obtain can be seen as the analogy of **??** in the case of the Hopfield network.

Lastly, we can plot the evolution of the mean of maximum overlaps across all replicas $\langle m^* \rangle$ depending on the storage load, see Figure 10. As expected, for the lowest values of $\alpha$, the mean maximum overlap quickly increases to $\langle m^* \rangle \approx 1$ and remains stable. The plateau value decreases as the storage load increases: it is less and less likely for the system to end up in a completely pattern-aligned configuration from a random initialization when the number of spurious states increases.

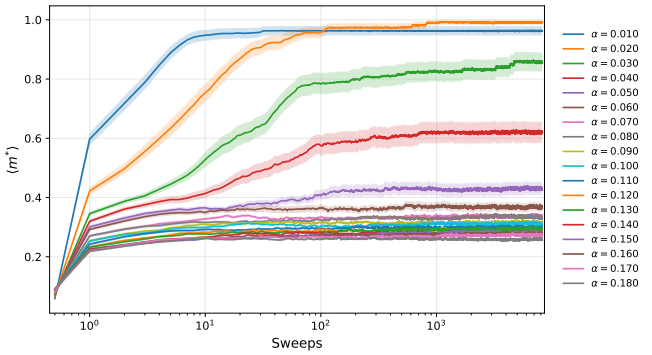



Figure 10: Evolution of $\langle m^* \rangle$ with the storage load.

### D. Performance test

We present in Figure 11 the performance analysis of the code obtained with the implementation presented in **??**. Several values of $\alpha$ and lattice sizes have been tested for an evolution of $2^{10}$ sweeps each. Subfigure (c) confirms that both implementations have complexities $\mathcal{O}\left(N \cdot N_{\text{neighbors}}\right) \sim \mathcal{O}\left(N^2\right)$ per sweep.

Compared to the two previous models, the speedup factor strikingly decreases close to 1 (and in some cases even below) in the case of the dense Hopfield network. The bitwise implementation is no longer more efficient than the reference one. In addition, the speedup factor appears to be mostly uncorrelated with the temperature and the storage capacity of the network.

The main explanation for this drop in efficiency of the bitwise implementation is the topology and the structure of the interactions in the case of the Hopfield model. Indeed, the Hopfield network is most commonly defined as a dense (fully connected) network, meaning that every neuron interacts with every other neuron in the network. This contrasts with the two previous models, in which each spin interacted with only four neighbors; in the Hopfield network, each neuron interacts with all $N - 1$ remaining neurons. Thus, if the loop over neighbors at each spin update is not perfectly optimized in the bitwise implementation, the loss in performance can become considerable for dense networks, as observed here. The loss in performance is however much more discrete in the case of the Ising and spin glass models, for which we only sum on the 4 closest neighbors each time.

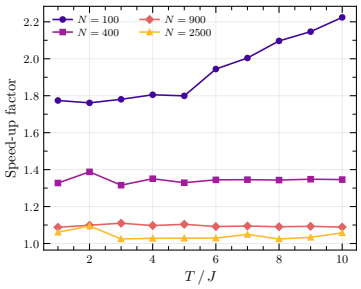
This hypothesis is confirmed by the decrease of the speedup factor with the lattice size, impacting the number of neighbors to sum over. Also, when testing the code on a sparse Hopfield model, the obtained speedup is very comparable to that of the other models. Conversely, when initialized on a dense network, the Ising model bitwise implementation also yields a significantly reduced speedup factor, confirming again our hypothesis. This is also reassuring, as it shows that the issue does not stem from the generalized use of `UnsignedInt` in this new bitwise implementation, but rather a common lack

of optimization in all models, that finds its sources in the coding of the base classes `Bits` and `UnsignedInt`.

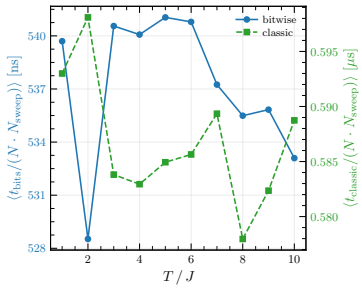
The main reason for this lack of efficiency in the neighbors loop might the creation of temporary `Bits` objects over the summations, killing the code performance when done hundreds of thousands ($\sim N^2$) times per sweep.

Two other related causes can explain the lack of performances for dense networks. The first one is the data structure and the other one is the memory access. The structure has indeed initially been built for sparse networks, with the use of lists of neighbors instead of direct interaction matrices. This implementation is effective when the network is sparse but becomes inefficient in the case of a dense network: instead of storing a fully filled $N \times N$ triangular interaction matrix containing $N(N-1)/2$ values (taking advantage of the symmetry), we store twice as many values in $N$ lists of $N-1$ components, thus increasing the memory usage, and potentially reducing the performance if the memory overflows the available RAM.

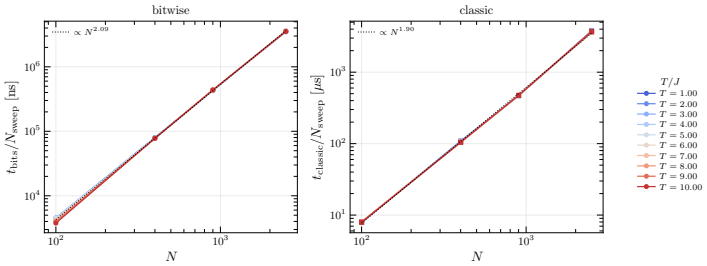
### E. Comparison of the two methods



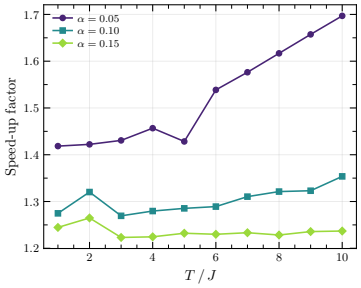

(a) Speedup factor evolution.



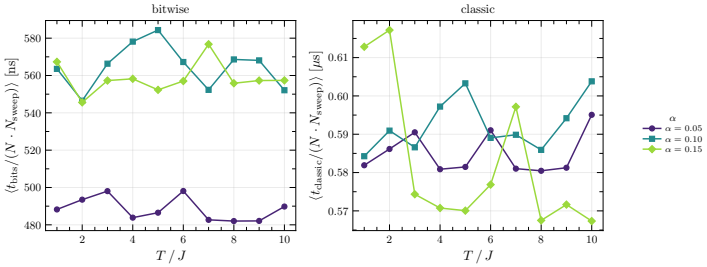

(b) Normalized time evolution.




(c) Sweep time depending on the lattice size for all temperatures.




(d) Speedup factor depending on the temperature for different $\alpha$ values.




(e) Sweep time depending on the temperature for different $\alpha$ values.

Figure 11: Performance analysis of the bitwise and classical Hopfield Metropolis implementations. In Subfigures (a), (b) and (c), the data has been averaged over the different values of $\alpha$.

## V. DISCUSSION

## VI. CONCLUSION

In this internship project under the supervision of Michele Castellana and David Lacoste, we implemented a bitwise version of the Metropolis algorithm. This version uses the machine-specific binary arithmetic to efficiently simulate the evolution of $n_{\text{bits}}$ systems in parallel. On top of this structural optimization, the same sequence of random numbers is used to make all $n_{\text{bits}}$ systems evolve. This turns out to be central as random number generation is often the simulation bottleneck. Using this bitwise formalism considerably reduces the number of operations required to make the system evolve, as all configurations are stored using `Bits` objects, whose evolutions are given by bitwise operations that are straightforward for the machine. For now, we use $n_{\text{bits}} = 64$, but this number can potentially be increased up $n_{\text{bits}} = 512$, providing a change in the type of objects used and low-level implementation of the code. The limitations of the size of $n_{\text{bits}}$ are directly related to the constraints of the CPU architecture.

For now, the code only provides a bitwise implementation of the Metropolis algorithm, but other Monte-Carlo algorithm such as the Glauber algorithm [14] or cluster flip algorithms [15, 16] can also be setup in this formalism. So far, the code enables the application of this bitwise Metropolis algorithm on several networks models: the Ising model, the Edwards-Anderson spin glass model and the Hopfield model. It can however be generalized to every model that uses discrete values, as it is always possible to shift those values into positive integers that are suitably stored in `UnsignedInt` objects.

The interest of this bitwise formalism is to efficiently sample the dynamics of a given system. Indeed, the Metropolis algorithm becomes computationally demanding when one attempts to sample a large number of configurations. This situation occurs, for example, when looking for rare samples in the dynamics, or sampling a large number of initial conditions.

This implementation therefore seemed ideal for studying the dynamics of the Hopfield model. Indeed, while the Hopfield model – introduced in its now-canonical form by Hopfield's landmark 1982 paper [3] – has become a reference in the statistical mechanics of networks and a milestone in research on both biological and artificial neural networks, its dynamical properties remain for the most part to be discovered. The objective of this internship was thus to use this implementation to study the dynamics of the model.

The code has first been tested on the Ising and Edwards-Anderson models, for which it reproduced the correct expected results, see **??** and **??**. In addition, it has proven to be efficient, yielding computational gains – that is the ratio of the number of independent results obtained for a given computation time between the optimized bitwise implementation and the reference one – comprised between 20 and 45, depending mostly on the temperature of the system. However, when applying the same bitwise implementation on the Hopfield network, the computational gain vanished. This is most likely due to the density of the network: one has to go through every single neuron in the network to update a single neuron. If the bitwise implementation creates some temporary `Bits` or `UnsignedInt` objects during the spin interaction calculations, the loss in performance can become considerable for dense networks. Fixing this low-level implementation in order to prevent the creation of temporary objects will be the focus of the remaining two weeks of this internship. Despite this loss of computational advantage, the bitwise implementation still yielded the expected results, see Figure 7. In addition, this implementation allows us to easily visualize the evolution of the $n_{\text{bits}}$ independent system in parallel, see **??**, Figure 8 and Figure 9.

This method thus proved promising to study the dynamics of the Hopfield network, that is the dynamical exponent, correlations or even the dependence of this dynamics on the network's structure. Unfortunately, this project will most likely not be pursued into a PhD project, due to the lack of fundings.


[1] The Royal Swedish Academy of Sciences. Press release: The Nobel Prize in Physics 2024. `https://www.nobelprize.org/prizes/physics/2024/press-release/`, October 2024. Accessed: June 2026.

[2] Shun-ichi Amari. Learning patterns and pattern sequences by self-organizing nets of threshold elements. *IEEE Transactions on Computers*, C-21:1197–1206, 1972.

[3] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.

[4] Daniel J. Amit, Hanoch Gutfreund, and H. Sompolinsky. Spin-glass models of neural networks. *Phys. Rev. A*, 32:1007–1018, Aug 1985.

[5] Daniel J Amit, Hanoch Gutfreund, and H Sompolinsky. Statistical mechanics of neural networks near saturation. *Annals of Physics*, 173(1):30–67, 1987.

[6] Daniel J. Amit. *Modeling Brain Function: The World of Attractor Neural Networks*. Cambridge University Press, Cambridge, 1989.

[7] Haim Sompolinsky. Statistical mechanics of neural networks. *Physics Today*, 41(12):70–80, 1988.

[8] John Hertz, Anders Krogh, and Richard G. Palmer. *Introduction to the Theory of Neural Computation*. Santa Fe Institute Studies in the Sciences of Complexity. Addison-Wesley, Redwood City, CA, 1991.

[9] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York, 1949.

[10] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[11] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 2023.

[12] H Freund and P Grassberger. Multispin coding for spin glasses. *Journal of Physics A: Mathematical and General*, 21(16):L801, aug 1988.

[13] Matteo Palassini and Sergio Caracciolo. Monte carlo simulation of the three-dimensional ising spin glass, 1999.

[14] Roy J. Glauber. Time-dependent statistics of the Ising model. *Journal of Mathematical Physics*, 4(2):294–307, 1963.

[15] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in Monte Carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987.

[16] Ulli Wolff. Collective Monte Carlo updating for spin systems. *Physical Review Letters*, 62(4):361–364, 1989.